

Interrupt Service Routines (ISR) in C++ and Embedded Systems

What is an ISR?

An Interrupt Service Routine (ISR) is a function triggered automatically in response to an interrupt signal from hardware or software. ISRs are used for handling real-time events like timer overflows, pin changes, serial data reception, etc.

Key Characteristics of ISRs

- Triggered by hardware (e.g., GPIO, Timer, UART)
- Must be non-blocking and fast
- Declared as `void`, no parameters
- No return values
- Avoid dynamic memory or I/O functions

Example: ISR on Arduino (C++)

```
volatile int count = 0;
```

```
void IRAM_ATTR handleInterrupt() {  
    count++; // Fast and minimal  
}
```

```
void setup() {  
    pinMode(2, INPUT_PULLUP);  
    attachInterrupt(digitalPinToInterrupt(2), handleInterrupt, FALLING);  
}
```

```
void loop() {  
    if (count > 0) {  
        // Handle the event  
        count = 0;  
    }  
}
```

Interrupt Service Routines (ISR) in C++ and Embedded Systems

```
}  
  
}
```

What Not to Do in ISR

- Don't use delay()
- Don't allocate memory (e.g., malloc or new)
- Don't use Serial.print or printf
- Don't perform long computations or I/O
- Don't access non-volatile shared variables unsafely

Best Practices

- Keep ISR minimal and fast
- Use `volatile` for shared variables
- Use flags or counters to communicate with main loop
- In RTOS, notify tasks using semaphores or queues from ISR

ISR in RTOS (FreeRTOS Example)

```
void IRAM_ATTR gpio_isr_handler(void* arg) {  
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;  
    xSemaphoreGiveFromISR(xSemaphore, &xHigherPriorityTaskWoken);  
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);  
}
```